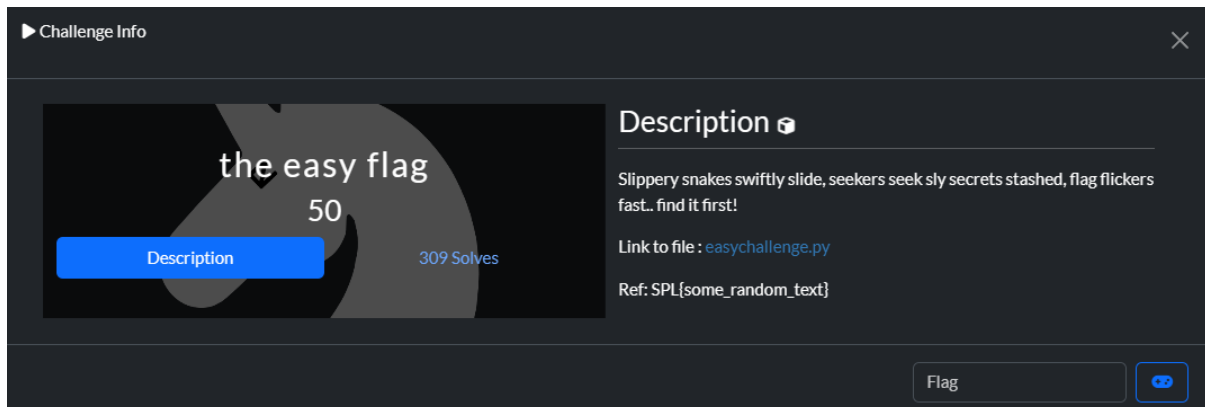


Challenge Name: the easy flag - 50 (MISC)



The screenshot shows a 'Challenge Info' window. On the left, there's a dark card with the title 'the easy flag' and the number '50'. Below the title is a blue button labeled 'Description' and the text '309 Solves'. On the right, the 'Description' section contains the text: 'Slippery snakes swiftly slide, seekers seek sly secrets stashed, flag flickers fast.. find it first!'. Below this, it says 'Link to file : [easychallenge.py](#)' and 'Ref: SPL{some_random_text}'. At the bottom right, there is a 'Flag' input field and a button with a Discord icon.

TL;DR

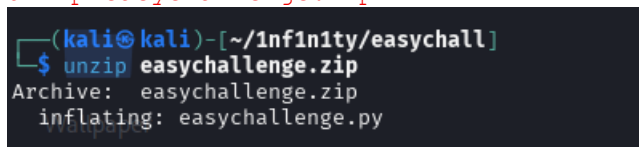
The provided Python file was heavily obfuscated but contained a compressed/encoded payload. I extracted and ran a small extractor ([solution.py](#)) which decompressed the hidden script and printed the flag.

Flag: `SPL{spl_is_cr4zy}`

Steps (reproducible)

1. Unzip the challenge

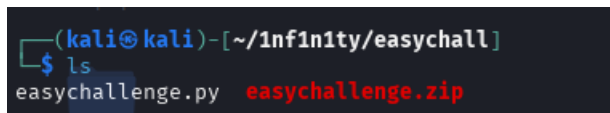
```
unzip easychallenge.zip
```



A terminal window showing the command `unzip easychallenge.zip` being executed. The output shows 'Archive: easychallenge.zip' and 'inflating: easychallenge.py'.

2. Verify files

```
ls
```



A terminal window showing the command `ls` being executed. The output shows two files: `easychallenge.py` and `easychallenge.zip`.

3. Create the extractor (solution.py)

Save this exact extractor as `solution.py` in the same folder:

```
(kali㉿kali)-[~/1nf1n1ty/easychall]
$ nano solution.py
(kali㉿kali)-[~/1nf1n1ty/easychall]
$ cat solution.py
#!/usr/bin/env python3
# short_extract.py - compact extractor

import re, sys, zlib, subprocess
from pathlib import Path

IN = Path(sys.argv[1]) if len(sys.argv) > 1 else Path("easychallenge.py")
OUT = Path("/tmp/easy_decoded.py")
if not IN.exists():
    sys.exit(f"ERROR: input {IN} not found")

txt = IN.read_text(errors="ignore")
cand = sorted([h for h in re.findall(r"[0-9a-fA-F]{8,}", txt) if h.startswith(("78", "789c"))],
               key=len, reverse=True)

if not cand:
    sys.exit("ERROR: no suitable hex blob found")
hexstr = cand[0]
blob = bytes.fromhex(hexstr)
try:
    src = zlib.decompress(blob)
except Exception as e:
    sys.exit(f"ERROR decompressing: {e}")

OUT.write_bytes(src)
print(f"WROTE: {OUT} ({len(src)} bytes)\n— preview —")
print("\n".join(src.decode("utf-8", "replace").splitlines()[:12]))
print("— end preview —\n")

# try run (time-limited) and show output
try:
    p = subprocess.run([sys.executable, "-I", str(OUT)], capture_output=True, text=True, timeout=6)
    print("== STDOUT ==\n", p.stdout)
    print("== STDERR ==\n", p.stderr)
    print("== RC:", p.returncode, "==\n")
except subprocess.TimeoutExpired as t:
    print(f"ERROR: execution timed out; partial stdout:\n{t.stdout}\npartial stderr:\n{t.stderr}")

(kali㉿kali)-[~/1nf1n1ty/easychall]
$
```

Example output from running the extractor (truncated for clarity):

[illegible]

Why it worked (short)

- `easychallenge.py` hid a large hex blob that, when decoded, yielded a zlib-compressed Python script.
- The extractor decompressed that blob and executed/printed the hidden content.
- Running the extracted code produced the flag string.

Final

Compact and classic — an obfuscated script with an embedded zlib blob. Extract, run, done.

Flag: `SPL{spl_is_cr4zy}`

`~By: 1nf1n1ty`